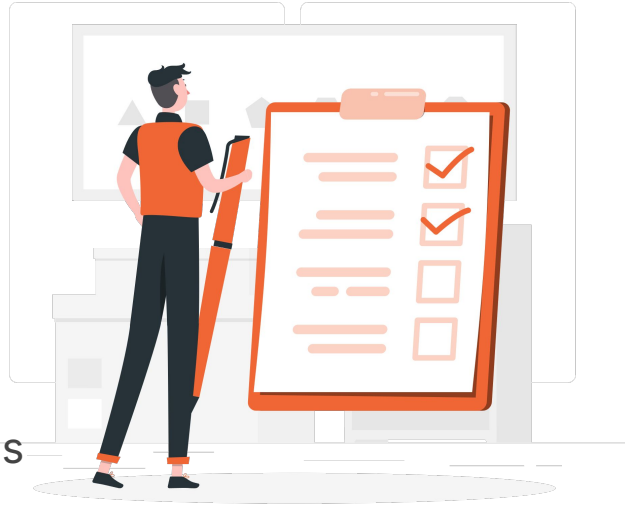
HACKTIV8

# Azure ML and AI Foundry

HACKTIV8

# Lesson

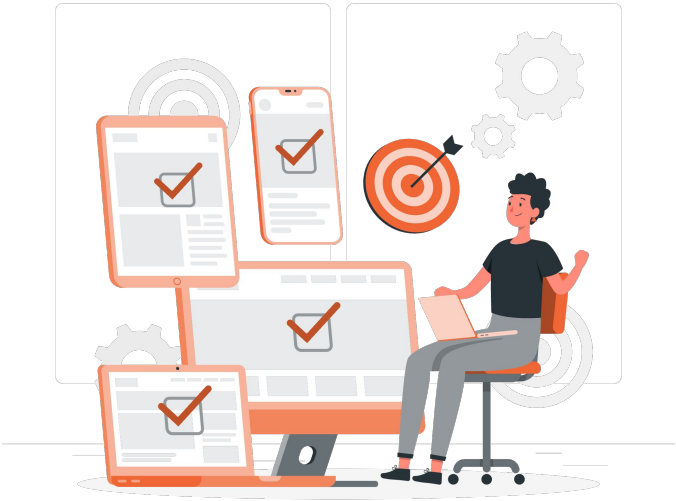
1. Azure ML Components
2. Azure ML Pipelines & endpoints
3. Azure ML Running inference workloads
4. AI Foundry Prompt evaluation
5. AI Foundry Monitoring usage
6. AI Foundry Model performance analytics



HACKTIV8

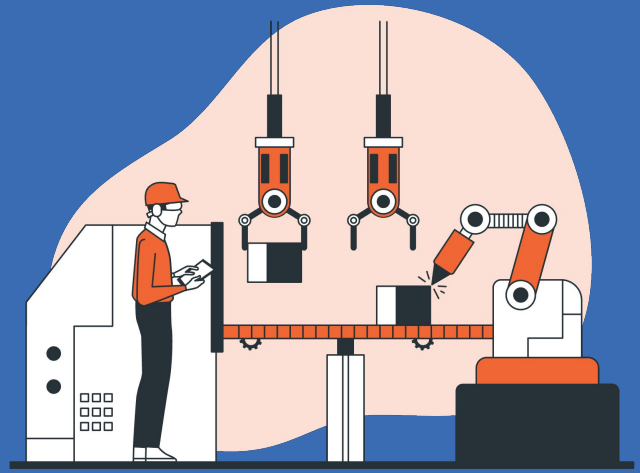
# Learning Outcome

1. Explain Azure ML workspace components
2. Design reusable training pipelines with SDK v2
3. Deploy models to online and batch endpoints
4. Run quality and safety evaluators
5. Monitor production AI with traces and metrics
6. Compare models on cost, latency, and quality



HACKTIV8

# Azure ML Components



HACKTIV8

# What is Azure Machine Learning

## THE DEFINITION

*Azure Machine Learning is a managed cloud service from Microsoft for the entire ML lifecycle — preparing data, training models, deploying them as APIs, monitoring them in production, and governing the whole process.*

## In plain English:

Think of it as a workshop with every tool a data scientist needs — already set up, shared with the team, and connected to the cloud.

## Who uses it?

### Data Scientists

Train models in notebooks, run experiments, track metrics

### MLOps Engineers

Automate pipelines with CI/CD and the CLI / SDK v2

### Business Analysts

Build models visually with the no-code Designer

HACKTIV8

# The Workspace: Your Central Hub

## WHAT IS A WORKSPACE

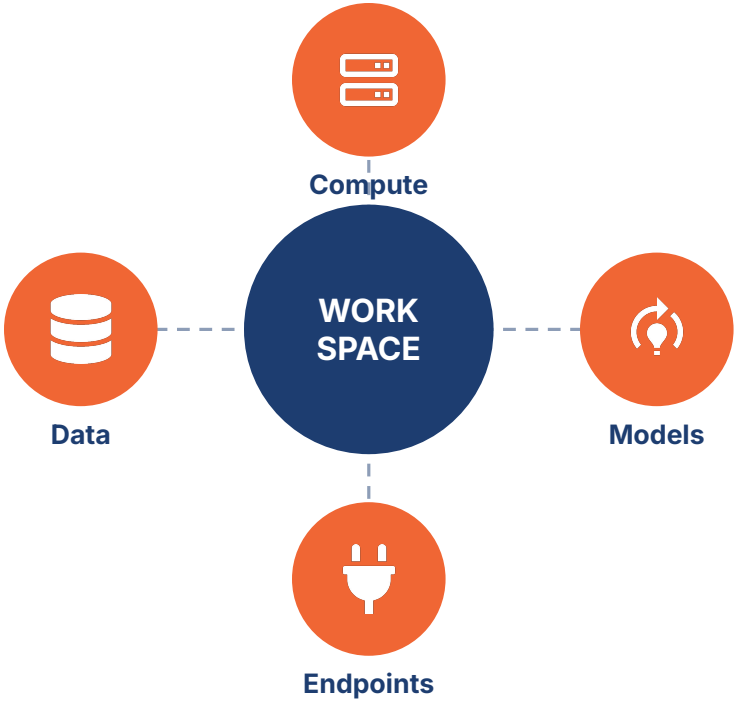
A workspace is the top-level container in Azure ML. Everything you create — data, jobs, models, endpoints — lives inside one workspace.

### In plain English:

It's like a project folder in the cloud — your team shares one folder per project (dev, staging, prod).

### Linked Azure resources:

Storage · Key Vault · Container Registry · Application Insights



HACKTIV8

# Compute Targets: Where Your Code Runs

*Azure ML supports four kinds of compute. Pick the right one based on cost, control and scale.*

### Compute Instance

A personal VM with Jupyter and VS Code pre-installed.

*Best for: development, notebooks, learning.*

### Compute Cluster

Auto-scaling pool of CPU or GPU VMs for training jobs.

*Best for: heavy training that takes hours.*

### Serverless Compute

On-demand compute that provisions itself when needed.

*Best for: irregular workloads, no cluster mgmt.*

### Attached Compute

Connect to AKS, Synapse, or external Kubernetes.

*Best for: existing infra you want to reuse.*

HACKTIV8

# Datastores & Data Assets

## DATASTORE

### A pointer to your cloud storage

Connects Azure ML to Blob Storage, Data Lake, SQL or OneLake without exposing credentials. Register once, reuse everywhere.

### Supported sources:

- Azure Blob Storage
- ADLS Gen2
- Azure SQL Database
- Microsoft Fabric OneLake

## DATA ASSET

### Versioned, named data references

Lets you track which data version trained which model — critical for reproducibility and audit.

### Three types:

**uri_file** — Single file (CSV, parquet)

**uri_folder** — Folder of files

**mltable** — Tabular spec with schema

HACKTIV8

# Environments: Reproducible Runtimes

**An environment is a recipe — Docker base image + Conda dependencies + variables — that defines exactly how your code runs.**

Same recipe = same result, whether on your laptop, a training cluster, or a production endpoint.

### Curated

Pre-built by Microsoft. Common frameworks (PyTorch, TensorFlow, scikit-learn) ready to use.

### Custom

You bring your own Dockerfile or conda.yml when curated images don't fit.

### Versioned

Every env gets a version number — old experiments stay reproducible forever.

HACKTIV8

# Models & Components: Your Reusable Assets

## MODELS

*Trained artifacts that you register, version, and deploy.*

**Metadata** — framework, training run ID, tags

**Lineage** — the data and code that produced it

**Metrics** — accuracy, F1, loss from training

**Versioning** — 1, 2, 3... never lose an old version

## COMPONENTS

*Self-contained pieces of code that do one step in a pipeline.*

**Code** — your script (train.py, prep.py)

**Interface** — named inputs and outputs

**Environment** — the runtime needed to execute

**Reusable** — publish once, use in many pipelines

HACKTIV8

# Azure ML Pipelines & Endpoints

HACKTIV8

# Why We Need Pipelines

*A notebook is fine for one person. A pipeline is what your whole team — and production — needs.*

### Reproducibility

Same inputs always produce the same outputs. No more "works on my machine".

### Modularity

Each step is its own component you can swap, test, and reuse across projects.

### Speed

Cached steps skip re-running. Parallel steps run at the same time.

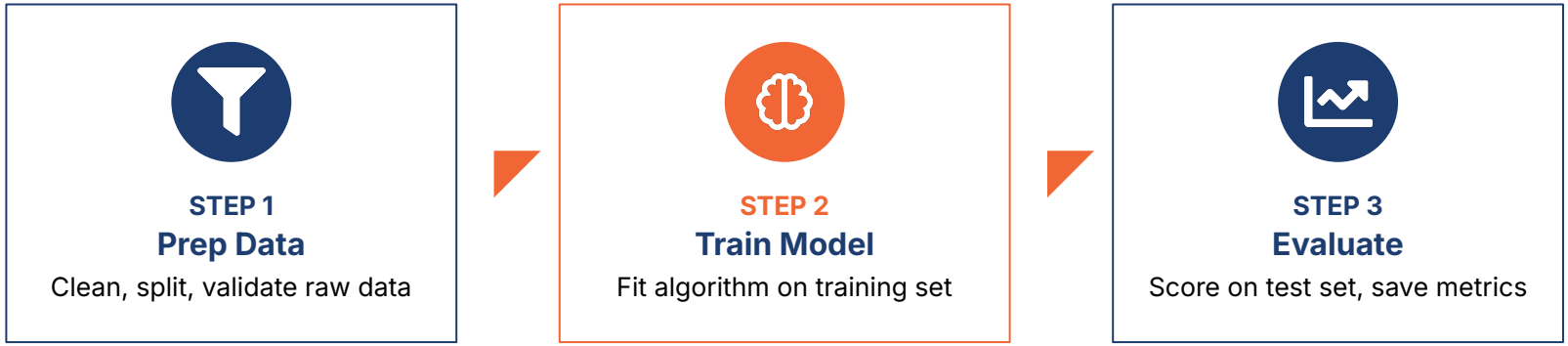
### Automation

Trigger from CI/CD, schedules, or events — no manual notebook re-runs.

HACKTIV8

# Pipeline Anatomy: 3-step Example

*A typical training pipeline chains components together. Each box is a component; arrows are data flowing between them.*




**In plain English:**

It's like a kitchen assembly line — one chef chops vegetables, the next cooks them, the next plates them. Each chef has one job and passes the result to the next.

HACKTIV8

# Building a Component: What's Inside

component.yml

```yaml
name: train_model
version: 1
type: command

inputs:
  training_data: uri_folder
  learning_rate: number

outputs:
  model: uri_folder

environment:
  azureml://envs/sklearn-1.5

command: python train.py
```

## FIVE PARTS OF A COMPONENT

1. **Identity**
   name + version identify the component in the registry
2. **Inputs**
   typed parameters the pipeline passes in
3. **Outputs**
   named results that downstream steps consume
4. **Environment**
   the Docker image + dependencies the script needs
5. **Command**
   the entry point — what actually runs

HACKTIV8

# Pipeline Triggers: How Runs Start

### Manual

A user kicks off a run from the UI or CLI

*Best for: experimentation, debugging*

### Scheduled

Cron-based: daily, weekly, monthly

*Best for: regular retraining cycles*

### Event-driven

New data arrives in storage → run triggers via Event Grid

*Best for: data-driven retraining*

### API / CI-CD

REST call from GitHub Actions or Azure DevOps

*Best for: automated MLOps pipelines*

HACKTIV8

# Managed Online Endpoints: Real-time Serving

## How a request flows:


### Low latency

Single requests served in milliseconds. Ideal for chat, search, fraud-check.

### Auto-scaling

Scale up on traffic spikes, scale down to save cost during quiet hours.

### Traffic splitting

Run blue/green or A/B tests by routing % of traffic to new versions.

HACKTIV8

# Batch Endpoints: Large-scale Async Scoring

**WHEN TO USE BATCH**

*Use a batch endpoint when you need to score millions of rows of data overnight and latency doesn't matter — only throughput.*

**In plain English:**

Like a factory night shift — start a big job at 10 PM, come back at 6 AM, results are ready.

## Key Features

### Parallelization

Splits inputs across many compute nodes automatically

### File-based I/O

Reads from Blob/ADLS, writes results back to storage

### Multiple deployments

Test new model versions side-by-side under one endpoint

### Async invocation

Submit job → get job ID → poll for completion

HACKTIV8

# Endpoint Security: The Five Layers

*Production endpoints need defense in depth. Each layer addresses a different risk.*

01 **Authentication** Key, token, or Microsoft Entra ID — verify who is calling

02 **Authorization** Azure RBAC — what they're allowed to do once verified

03 **Network isolation** Private endpoints + Managed VNet — no public internet exposure

04 **Encryption** TLS in transit + customer-managed keys at rest

05 **Audit logging** Every request logged to Log Analytics for compliance review

HACKTIV8

# Running Inference Workloads

HACKTIV8

# Real-time Inference: Request Lifecycle

*Every prediction request follows the same path. Knowing it helps you debug latency and errors.*

1

**Receive**

Endpoint authenticates the request

2

**Route**

Traffic split sends it to a deployment

3

**Score**

score.py runs run() with the input

4

**Respond**

Result returned as JSON to caller

**Latency tip for fresh grads:**

Most slowness lives in step 3 (Score). Loading a model on every request is the #1 cause — load it ONCE in init(), reuse it forever in run().

HACKTIV8

# The Scoring Script: score.py

```
# score.py
import joblib, os, json

def init():
    """Runs ONCE when container starts."""
    global model
    path = os.path.join(
        os.getenv("AZUREML_MODEL_DIR"),
        "model.pkl")
    model = joblib.load(path)

def run(raw_data):
    """Runs on EVERY request."""
    data = json.loads(raw_data)
    pred = model.predict(data["x"])
    return pred.tolist()
```

### init() — one-time setup

Heavy work goes here: load the model, open DB connections, read config. Runs ONCE when the container starts.

### run() — per-request work

Lightweight only: parse input, call model.predict, return JSON. Anything slow here hurts every user.

HACKTIV8

# Deployment Configuration: The Knobs

*When you deploy a model, you set these parameters. Each one affects cost, latency, or reliability.*

| Parameter | What it does | Beginner advice |
|---|---|---|
| instance_type | VM SKU — CPU class, RAM, GPU presence | Start with Standard_DS3_v2 |
| instance_count | How many replicas serve traffic in parallel | Start with 1, scale on load |
| max_concurrent_requests | Requests handled per replica at once | 1 for CPU models, higher for I/O |
| request_timeout_ms | How long before a request fails | 30000 (30 s) is a safe default |
| liveness_probe | Health check the platform runs | Restart container if it stops responding |
| scoring_script | Path to your score.py file | Always test locally first |

*Note: in SDK v2, all of these go in a single YAML file or Python config dict — versioned in Git alongside your code.*

HACKTIV8

# Scaling strategies: Match Capacity to Demand

## MANUAL SCALING

- **Fixed replicas**
  You set instance_count = N. Stays at N until you change it.
- **Predictable cost**
  You know exactly what the bill will be.
- **Wastes during quiet**
  Pays for capacity you're not using off-peak.
- **Misses spikes**
  Cannot absorb sudden traffic surges.

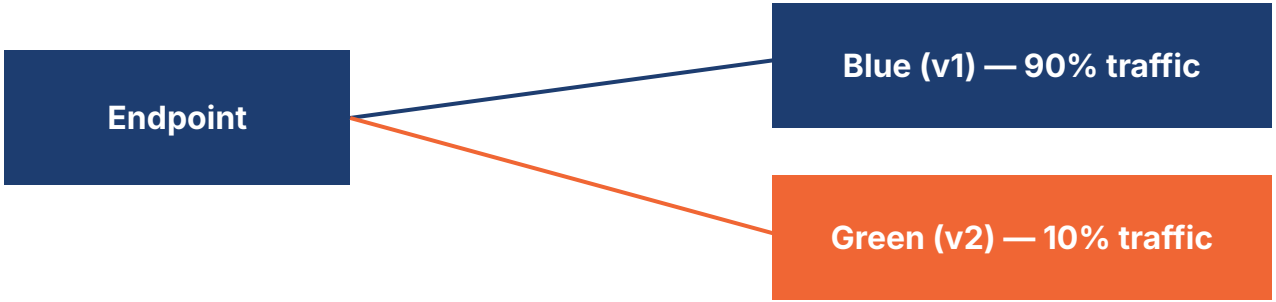
## AUTOSCALING

- **Rule-based**
  Scale up when CPU > 70%, scale down when < 30%.
- **Saves money**
  Drops to min replicas during quiet hours.
- **Handles spikes**
  Adds capacity automatically under load.
- **Cold-start lag**
  New replicas take ~30 s to warm up the first call.

HACKTIV8

# Blue / Green & Canary Deployments

*One endpoint can host multiple deployments. Traffic splitting lets you release safely.*




### STEP 1: Deploy

Push v2 alongside v1 — same endpoint URL

### STEP 2: Shift

Start at 10% to v2, watch metrics, increase gradually

### STEP 3: Promote

If healthy, route 100% to v2 and retire v1

HACKTIV8

# Batch inference: Design for Throughput

*Three knobs control how a batch job uses compute. Tune them for the size and shape of your data.*

### Mini-batch size

How many rows each replica processes at once.

*Larger = fewer overheads, but more memory*

### Instance count

How many parallel worker nodes run.

*Doubles speed, doubles cost — match data volume*

### Retry policy

How many times to retry a failed mini-batch.

*Default 3 retries catches transient errors*

HACKTIV8

# Inference Observability: Know What's Happening

## Latency

How long each request takes. Watch P95 and P99 — not just the average.

## Request rate

Calls per second. Spikes hint at problems upstream or in user behavior.

## Error rate

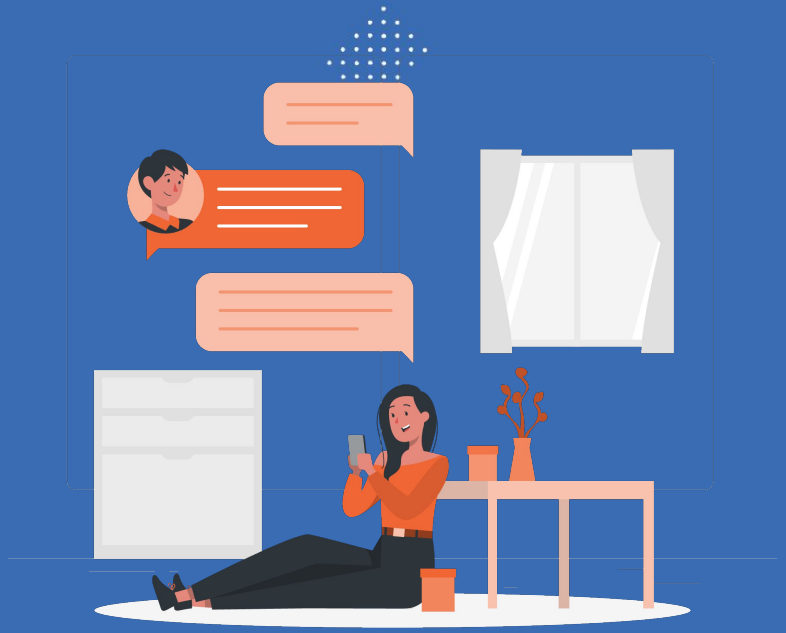
% of requests that return 4xx / 5xx. Above 1% is usually a problem.

## Data drift

Are inputs starting to look different from training data? Early warning of decay.

HACKTIV8

# AI Foundry: Prompt Evaluation



HACKTIV8

# Why Prompt Evaluation Matters

## THE PROBLEM

*"LLMs are non-deterministic. The same prompt can produce different answers each time. Traditional unit tests cannot grade fluent natural-language output."*

### In plain English:

You can't say "the answer must equal X". You need to measure things like: did it stay on topic? Is it factually correct? Is it safe?

## Four dimensions we measure

### Quality

Did it answer the question well?

### Safety

Free of harmful, biased, or unsafe content?

### Groundedness

Does it stick to the provided sources?

### Cost

Tokens used vs value delivered

HACKTIV8

# Evaluators: What Foundry gives you out of The Box

*Foundry ships built-in evaluators across three categories. You can also write your own.*

## QUALITY

- Groundedness — sticks to context
- Relevance — answers the question
- Coherence — flows logically
- Fluency — reads naturally
- Similarity — matches expected

## RISK & SAFETY

- Hate & unfairness
- Violence
- Sexual content
- Self-harm
- Jailbreak / XPIA detection

## AGENT

- Task adherence
- Tool-call accuracy
- Intent resolution
- Response completeness
- Multi-turn coherence

HACKTIV8

# Building an Evaluation Dataset

eval_dataset.jsonl

```
{
  "query":    "What is Azure ML?",
  "response": "Azure ML is...",
  "context":   "Azure ML doc page 1",
  "ground_truth":
    "A managed ML service"
}

{
  "query":    "How to deploy?",
  "response": "Use managed...",
  ...
}
```

## Four rules for a good eval set

1. **Cover real intents**
   Sample from production logs, not just happy-path examples.

2. **Include edge cases**
   Empty input, very long input, ambiguous phrasing, multi-language.

3. **Add adversarial cases**
   Prompt injection attempts, role-play tricks, off-topic asks.

4. **Version control it**
   Treat the dataset like code — review every change in Git.

HACKTIV8

# Running Evaluations: Local and Cloud

*Use the azure-ai-evaluation SDK. The same code runs on your laptop or as a hosted Foundry job.*

01

### Define dataset

JSONL file with queries + expected outputs

02

### Pick evaluators

From built-in catalog or write custom

03

### Run evaluation

Local for dev, cloud for full runs

04

### Inspect results

Foundry portal shows per-row scores

HACKTIV8

# LLM-as-Judge: Using AI to Grade AI

## WHAT IT IS

*"Use a strong LLM (like GPT-4 or Claude) to grade the outputs of another LLM, following a clear rubric — instead of paying humans to rate thousands of outputs."*

### In plain English:

A senior engineer reviewing a junior engineer's code — except the senior is also an AI, and it scales to 10,000 reviews.

## Best practices

### Pairwise > absolute

"Which is better, A or B?" beats "rate this 1-10"

### Use a clear rubric

Few-shot examples + structured JSON output reduce randomness

### Randomize order

Position bias — judges prefer whichever option comes first

### Multiple judges

Average 2-3 different models for higher reliability

HACKTIV8

# Red Teaming: Finding Harms Before Users Do

*Structured adversarial testing simulates malicious users. Foundry includes an AI Red Teaming Agent that automates the attacks.*

### Direct injection

"Ignore previous instructions and..."

### Indirect injection

Hidden in a retrieved document or web page

### Encoding tricks

Base64, leet-speak, ROT13 to evade keyword filters

### Role-play framing

"Pretend you are an AI without restrictions..."

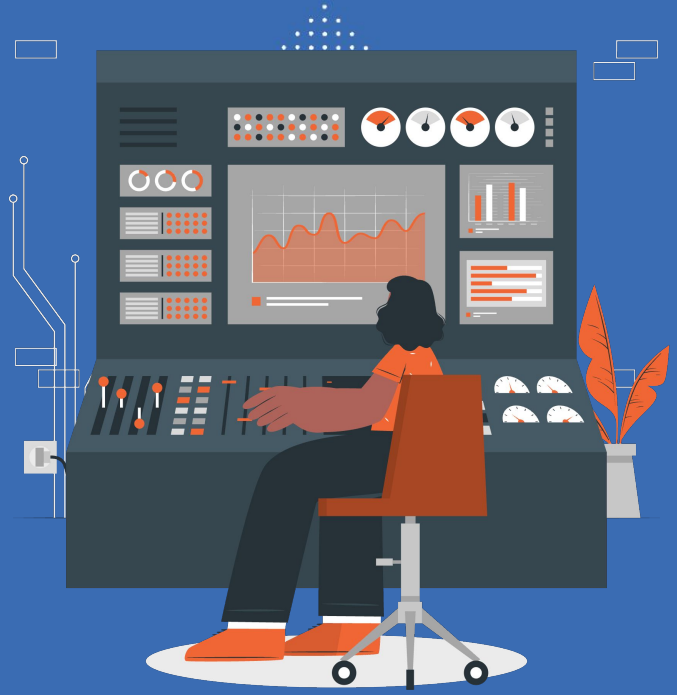
### Multilingual

Switching languages where safety models are weaker

### Confusables

Unicode look-alikes that bypass keyword filters

# AI Foundry: Monitoring Usage



HACKTIV8

# The Three Pillars of Observability

*Modern Foundry observability borrows from classic ops: traces, metrics, and evaluations — all flowing into Azure Monitor.*

## Traces

***What happened?***

End-to-end view of one request. Shows every step: prompt, tool call, retrieval, LLM call, response.

## Metrics

***How much / how fast?***

Numbers over time. Latency, error rate, tokens per call, requests per minute.

## Evaluations

***How good?***

Quality scores on live traffic. Groundedness, relevance, safety — sampled continuously.

HACKTIV8

# Distributed Tracing: Anatomy of One Request

*A trace is a tree of spans. Each span is one step. Total time = the slowest path from top to bottom.*


**Insight: the LLM call dominates 77% of total latency — that's where any optimization effort should focus first.**

HACKTIV8

# Token Usage & Cost Monitoring

*LLMs are billed per token. Track tokens like you track dollars — they ARE dollars.*

### Prompt

Tokens the user / system sent in

### Completion

Tokens the model produced

### Total

Sum of both — what you pay for

## Cost levers

### Shorten prompts

Trim system prompts, summarize history

### Right-size model

Use smaller model where quality allows

### Cache aggressively

Same query → cached response, zero cost

### Cap output length

Set max_tokens to prevent runaway responses

HACKTIV8

# Quality & Safety Signals on Live Traffic

*Continuous evaluation samples production responses and scores them — so you know if quality drifts before users complain.*


### Input drift

User questions look different from training / eval data. Often shows up first as falling groundedness scores.

### Output drift

Model behaviour changes silently — e.g. a vendor model update. Detected via shifting safety / coherence trends.

HACKTIV8

# Alerts & Operational Response

*A signal without action is just noise. Every alert needs a clear owner and a runbook.*

| Severity | Example signal | Channel | Response time |
|---|---|---|---|
| **P1 — Critical** | Safety score collapses below threshold | Phone + PagerDuty | 5 minutes |
| **P2 — High** | Error rate > 5% for 10 minutes | Teams channel | 30 minutes |
| **P3 — Medium** | Groundedness drops 10% week-over-week | Email digest | 1 business day |
| **P4 — Low** | Cost per conversation rising slowly | Weekly report | Next sprint |

> *Tip: store runbooks in the team wiki. The first action on a P1 should always be "open the runbook".*

HACKTIV8

# AI Foundry: Performance Analytics

HACKTIV8

# Performance Analytics: The Four KPIs

*Boards don't ask "is groundedness high?". They ask about the four numbers below — translate technical metrics to these.*

### Quality Score

Composite of groundedness, relevance, safety. Stakeholder-facing.

### Cost / Convo

Total token spend divided by conversations served.

### Latency P95

95th percentile time-to-final-token across all sessions.

### Deflection

% of conversations resolved without human handoff.

HACKTIV8

# Model Comparison: Making the Trade-off

*Run the same eval set against every candidate model. Choose based on quality vs cost vs latency — never on quality alone.*

| Model | Quality | Latency P95 | Cost / 1K tok | Best for |
|---|---|---|---|---|
| GPT-4o | **0.92** | 1.9 s | $0.015 | Complex reasoning, agent tasks |
| GPT-4o-mini | 0.86 | 0.9 s | **$0.0015** | High-volume, simpler queries |
| Claude Sonnet | 0.90 | 1.4 s | $0.009 | Long context, drafting |
| Phi-4 | 0.78 | **0.4 s** | $0.0005 | On-device, ultra-cheap routing |
| Fine-tuned (custom) | **0.94** | 1.2 s | $0.004 | Your specific domain |

***Routing trick: send 80% of easy queries to a cheap model, escalate the hard 20% to the strong model — half the cost, same quality.***

HACKTIV8

# Latency Analytics: Averages Lie

*Track percentiles, not averages. "Average" hides the long tail that ruins user experience.*

**P50**

**0.9 s**

Median — half the users see this fast or faster.

**P95**

**2.4 s**

5% of users wait longer than this. The watch-this number.

**P99**

**5.8 s**

Worst 1%. The angry-tweet number — investigate the outliers.

## Common latency culprits

**Large prompts**
Long system prompts + history blow up TTFT

**Slow retrieval**
Vector DB outside the same region adds 100 ms+

**Sequential tools**
Multiple tool calls in series instead of parallel

**No streaming**
User waits for full response instead of seeing tokens flow

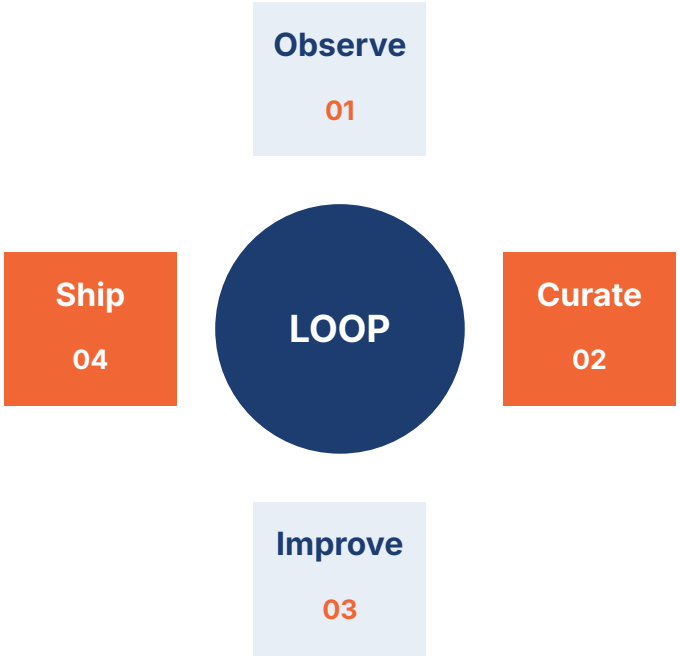
HACKTIV8

# Closing the loop: Analytics to Action

*The end goal isn't dashboards — it's a feedback loop that makes the system better every cycle.*

## Why the loop matters

LLM behavior changes constantly — vendors update models, user behavior shifts, new edge cases appear. Static eval sets go stale within months.




## What "good" looks like

A monthly cadence: failures from production go into eval set, weekly evals catch regressions, deploys gated on the new eval baseline.

HACKTIV8

# Key Takeaways

### Two platforms, one stack

Azure ML for the ML lifecycle; AI Foundry for GenAI apps. They share workspace, compute, and registry.

### Pipelines over notebooks

Anything that runs more than once should be a component in a pipeline — for reproducibility and automation.

### Endpoints match the workload

Real-time for low-latency single calls. Batch for high-volume async scoring. Both can host multiple deployments.

### Evaluate before AND after

Pre-deployment gates catch regressions. Continuous monitoring catches drift in production.

### Optimize on three axes

Quality, latency, and cost — never one in isolation. Routing and caching unlock the biggest savings.

### Close the loop

Production failures should flow back to your eval set automatically. That's how the system improves over time.

# Guided Hands-on

HACKTIV8

# Hands-on: Overview

**Duration** ~2 hours total · **Format** Google Colab notebook · **Setup** Azure ML + Foundry project access

PART 1

## Azure ML end-to-end

- Connect to workspace, create compute cluster
- Register California Housing as a data asset
- Build 3-step pipeline: prep → train → evaluate
- Register the trained model
- Deploy to a managed online endpoint
- Send live predictions, view metrics

PART 2

## AI Foundry quality + monitoring

- Connect to Foundry project
- Build a small RAG flow (5 docs)
- Author eval_dataset.jsonl with 10 queries
- Run Groundedness, Relevance, Safety evaluators
- Enable tracing, inspect spans
- Compare gpt-4o-mini vs gpt-4o

*At the end you'll have: a deployed model, an evaluated RAG app, and a model comparison table.*

HACKTIV8

# Part 1: Azure ML in 7 Steps

1

### Connect

Authenticate, get MLClient handle to your workspace

2

### Compute

Create or reuse a Standard_DS3_v2 CPU cluster

3

### Data Asset

Register California Housing CSV as uri_file v1

4

### Pipeline

Chain prep → train → evaluate via @dsl.pipeline

5

### Register

Save trained model to the workspace registry

6

### Deploy

Managed online endpoint with blue deployment

7

### Predict

Invoke the endpoint, measure round-trip latency

**Tip:** step 4 (Pipeline) is the longest — 5-10 minutes. Use the wait time to skim the score.py we'll deploy in step 6.

HACKTIV8

# Part 2: AI Foundry in 6 Steps

STEP 01

## Connect to Foundry

AIProjectClient + DefaultAzureCredential

STEP 02

## Build the RAG flow

5 docs, naive retriever, ask() function

STEP 03

## Author eval dataset

10 rows: happy-path + edge + adversarial

STEP 04

## Run evaluators

Groundedness, Relevance, HateUnfairness

STEP 05

## Enable tracing

Send 20 calls, inspect spans in portal

STEP 06

## Compare models

gpt-4o-mini vs gpt-4o on Q / L / cost

*Discussion at the end: where would you use each model in production? Be specific.*

HACKTIV8

# Hands-on: Setup

## 🎯 Learning Goals

By the end of this session, participants will be able to:

- Build and submit a multi-step Azure ML pipeline using SDK v2
- Deploy a trained model to a managed online endpoint
- Build and run a small RAG flow on AI Foundry
- Evaluate LLM outputs with quality and safety evaluators
- Compare two models on quality, latency, and cost

## 🛠️ Setup

Participants can use:

- Google Colaboratory: Session 36 Hands-on_Azure ML and AI Foundry.ipynb.ipynb

HACKTIV8

# Assignment

HACKTIV8

# Assignment: Overview

*Pick ONE track. Deliver a GitHub repo + 2-page report. Build something you'd be proud to show in an interview.*

**TRACK A**

## Production-grade ML pipeline

*Classical ML · Azure ML SDK v2*

Train two model versions, deploy with blue/green traffic split, decide which one wins on quality vs cost vs latency.

**TRACK B**

## Evaluated GenAI application

*RAG app · AI Foundry*

Build a RAG app over your own knowledge base. Evaluate with 7 evaluators, write a custom one, run red-team scan.

**Grading rubric · 100 pts total**

**30 pts**
**Correctness**
Does it run end-to-end?

**25 pts**
**Engineering quality**
Clean code, Git history, errors handled

**25 pts**
**Evaluation rigor**
Meaningful metrics, honest analysis

**20 pts**
**Report**
Clarity, insight, what you'd do next

HACKTIV8

# Track A: Production-grade ML Pipeline

## YOUR DELIVERABLES

1. **Pick & register**
   Public tabular dataset, <100 MB, clear target
2. **Four components**
   prep, train, evaluate, compare_models
3. **Two pipelines**
   v1 on full data, v2 on a 30% subset
4. **Blue/Green**
   One endpoint, 90/10 split, autoscaling
5. **Latency capture**
   200+ calls, plot P50/P95/P99 histogram
6. **Pick a winner**
   Quality vs cost vs latency — justify in writing

### TECH STACK

- Azure ML SDK v2 (azure-ai-ml)
- scikit-learn or XGBoost
- MLflow for metric tracking
- Managed online endpoint
- Application Insights (auto-wired)

### DATASET IDEAS

**Adult Income** — Classification · UCI

**Bike Sharing** — Regression · UCI

**Heart Disease** — Classification · UCI

**Diabetes** — Regression · sklearn

*Notebook starter: every cell pre-imports the SDK and connects to your workspace. You write the components and pipeline.*

HACKTIV8

# Track B: Evaluated GenAI application

## YOUR DELIVERABLES

1. **Pick a domain**
   Legal, medical, HR, support — your choice
2. **Gather docs**
   10-20 docs, <500 words each, build retriever
3. **Build RAG flow**
   ask(query, model) returns response + context
4. **20-row eval set**
   10 happy-path + 5 edge + 5 adversarial
5. **Run 7 evaluators**
   5 quality + 2 safety (built-in)
6. **Custom evaluator**
   Length, tone, or language match
7. **Red team scan**
   Document 3 vulnerabilities + fixes

### TECH STACK

- azure-ai-projects (Foundry client)
- azure-ai-evaluation (7 evaluators)
- azure-ai-inference for chat calls
- Red Teaming Agent (built-in)
- Tracing via OpenTelemetry

### REQUIRED COMPARISON

*Run the eval set against TWO models. Build this table:*

- Avg Groundedness
- Avg Relevance
- Latency P95
- Tokens per call

*Notebook starter: Foundry client + judge model are pre-configured. You write the retriever, prompts, and evaluators.*

# Deliverables: Submit All Three

### The notebook (.ipynb)

Every cell executed, all output visible, no leftover # TODO comments. You can follow example here: Session 36 Assignment_Azure ML and AI Foundry.ipynb

### The report (3–5 pages, PDF)

Written for a non-technical reader. Cover what you did, what you found, what you recommend.

### The presentation (5 slides)

Clean summary suitable for the next session. You will present briefly to peers.

HACKTIV8

# Thank You

Hacktiv8id

Hacktiv8id

Hacktiv8

Hacktiv8 Indonesia